CV Cheat Sheet | miniyuan

1 Visual Task Hierarchy

Sensation $\to$ Processing $\to$ Perception $\to$ Cognition. (1) Acquisition. RGB/depth cameras, LiDAR $\to$ images, video, point clouds, meshes. (2) Low-Level. Processing and feature extraction: denoising, deblurring, watermark removal, edges/corners. (3) Mid-Level. Real-world inference: 3D reconstruction, panorama stitching. (4) High-Level. Semantic representation and interpretation: scene understanding, AR.

2 Edge Detection

2.1 Conceptions

Image as Discrete Function

Image $I(x, y)$ is a 2D discrete function with **intensity** values.

LTI Systems

Linear time-invariant system satisfies: linearity $ax(t) \to ay(t)$, $x_1 + x_2 \to y_1 + y_2$; shift-invariance $x(t-\tau) \to y(t-\tau)$.

Convolution vs Cross-Correlation

2D convolution: $(f * g)[m, n] = \sum_k \sum_\ell f[k, \ell] g[m-k, n-\ell]$.

2D cross-correlation: $(f \star g)[i, j] = \sum_m \sum_n f[i+m, j+n] g[m, n]$.

Convolution flips kernel; cross-correlation does not. Convolution satisfies commutativity/associativity; cross-correlation does not.

2.2 Frequency Domain

Fourier Transform

Discrete: $\mathcal{F}\{f\}(\omega) = \sum_n f[n] e^{-i2\pi\omega n}$. Continuous: $\mathcal{F}\{f\}(\omega) = \int f(t) e^{-i2\pi\omega t} dt$. Convolution theorem: $\mathcal{F}\{f * g\} = \mathcal{F}\{f\} \cdot \mathcal{F}\{g\}$.

Gaussian Filtering

1D spatial: $g(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp(-\frac{x^2}{2\sigma^2})$. Frequency: $\mathcal{F}\{g\}(\omega) = \exp(-\frac{\sigma^2\omega^2}{2})$. 2D spatial: $g(x, y) = \frac{1}{2\pi\sigma^2} \exp(-\frac{x^2+y^2}{2\sigma^2})$. Larger σ selects more **low-frequency** components.

2.3 Edge Detection

Edge Gradients

Gradient vector: $\nabla f = (\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y})$. Magnitude: $|\nabla f| = \sqrt{(\frac{\partial f}{\partial x})^2 + (\frac{\partial f}{\partial y})^2}$. Orientation: $\theta = \operatorname{atan2}(\frac{\partial f}{\partial y}, \frac{\partial f}{\partial x})$.

Finite Difference and Derivative Theorem

Central difference: $\frac{\partial f}{\partial x} \approx \frac{f(m+1,n) - f(m-1,n)}{2}$. Derivative theorem of convolution: $\frac{\partial}{\partial x}(f * g) = f * \frac{\partial g}{\partial x}$. Efficient edge detection: $G_x = f * \frac{\partial g}{\partial x}$, $G_y = f * \frac{\partial g}{\partial y}$; $|\nabla f| = \sqrt{G_x^2 + G_y^2}$.

Canny Detector

Canny pipeline: (1) Gaussian derivative filtering for G_x, G_y; (2) compute magnitude and orientation; (3) NMS for single-pixel edges; (4) hysteresis thresholding with edge linking.

NMS

for pixel q, compare $M(q)$ with interpolated $M(q_1), M(q_2)$ along gradient direction; keep if largest. Hysteresis: $maxVal = 0.3 \times avg$, $minVal = 0.1 \times avg$. Strong ($M \geq maxVal$): keep. Weak ($minVal \leq M < maxVal$): keep if connected to strong. Weak suppressed otherwise.

3 Classical Feature Extraction

3.1 Line Fitting

Least Squares

Fit $y = kx + m$ by minimizing $\sum_{i=1}^n (y_i - (kx_i + m))^2$. Matrix form: $\min_{\boldsymbol{\theta}} \|A\boldsymbol{\theta} - \boldsymbol{y}\|_2^2$ with $A = [x_i, 1]$, $\boldsymbol{\theta} = [k, m]^T$. Solution: $\boldsymbol{\theta} = (A^T A)^{-1} A^T \boldsymbol{y}$. Unstable for near-vertical lines.

SVD Fitting

Improved: $\min_{a,b,d} \sum (ax_i + by_i - d)^2$ s.t. $a^2 + b^2 + d^2 = 1$. Matrix form: $\min_{\boldsymbol{h}} \|A\boldsymbol{h}\|^2$ with $A = [x_i, y_i, 1]$, $\|\boldsymbol{h}\| = 1$. SVD: $A = UDV^T$. Solution: $\boldsymbol{h} = \pm \boldsymbol{v}_3$ (right singular vector for smallest singular value).

RANSAC

Robust to outliers via random sampling. Steps: (1) choose s, δ, p; (2) sample s points; (3) fit model; (4) count inliers; (5) update best; (6) repeat N times; (7) refit on best inliers. Required iterations: $N = \frac{\log(1-p)}{\log(1-(1-e)^s)}$ where e is outlier ratio.

3.2 Corner Detection

Harris Response

Energy after shift (u, v): $E(u, v) \approx [u, v] M [x_0, y_0] [u, v]^T$. Structure tensor: $M = \sum_{(x,y) \in N} \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$, often with Gaussian weighting. Corner response: $\theta = \det M - \alpha(\operatorname{tr} M)^2 = (I_x^2 I_y^2 - (I_x I_y)^2) - \alpha(I_x^2 + I_y^2)^2$. $\theta \gg 0$: corner; $\theta \ll 0$: edge; $\theta \approx 0$: flat.

Harris Pipeline

Steps: (1) compute gradients I_x, I_y; (2) compute $I_x^2, I_y^2, I_x I_y$; (3) Gaussian smooth the three maps; (4) compute θ, threshold it; (5) NMS to keep local maxima.

Geometric Properties

Harris has the following properties: Translation equivariant: $T_{u,v}(\Theta(A)) = \Theta(T_{u,v}(A))$. Rotation equiv with **isotropic** filter: $R_\phi(\Theta(A)) = \Theta(R_\phi(A))$. Scale non-equiv: viewpoint changes break detection.

3.3 Classical Recognition Pipeline

Feature Detection

Keypoint detector (Harris, DoG) + descriptor (SIFT). SIFT: 128-dim vector from 4×4 spatial grids with 8 orientation bins each, after orientation normalization.

Bag-of-Visual-Words

Steps: (1) collect descriptors; (2) k-means cluster into K visual words; (3) assign each descriptor to nearest word (hard/soft); (4) build K-dim frequency histogram. Loses spatial information.

Limits of Classical CV

Manual feature design; error accumulation across stages; weak high-level semantics; poor scaling with data; limited generalization.

4 ML and Neural Network Basics

4.1 Problem Setup

Task Formulation

Given input $x \in \mathbb{R}^d$, predict label y. Soft output: $p(y = 1|x) \in [0, 1]$; hard: $p(y = 1|x) \geq \tau$. Model: $h(x; \theta)$ with parameters θ.

Data Pipeline

Split: train (learn, update params), validation (tune hyperparams), test (final eval), inference (deploy). Preprocessing: normalize (zero-mean/unit-var or $[0, 1]$); augment (crop, flip, rotate); batch and shuffle.

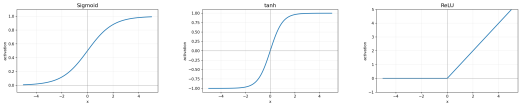
4.2 Models

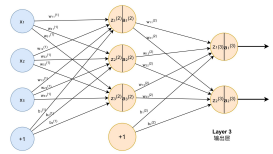
Logistic Regression

Linear score $z = \theta^T x + b$. Sigmoid: $\sigma(z) = \frac{1}{1+e^{-z}}$. Parameters: $\theta \in \mathbb{R}^d$, $b \in \mathbb{R}$.

MLP

Layer: $\mathbf{z}^{(\ell)} = \mathbf{W}^{(\ell)} \mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}$, $\mathbf{a}^{(\ell)} = g_\ell(\mathbf{z}^{(\ell)})$. Activations: ReLU $\max(0, x)$ (default); LeakyReLU $\max(\alpha x, x)$; Sigmoid $\frac{1}{1+e^{-x}}$; tanh.





orange circles (Neurons, memory footprint); directed edges (parameters, parameter count).

4.3 Objective

Loss Functions

Sigmoid: $\sigma(z) = 1/(1 + e^{-z})$, $p(y = 1|x; \theta) = \sigma(z_i) = h_i$; log-likelihood $\log p(Y|X; \theta) = \sum_{i=1}^n [y_i \log h_i + (1 - y_i) \log(1 - h_i)]$ (MLE maximizes it). Binary CE/NLL: $\mathcal{L} = -\frac{1}{n} \log p(Y|X; \theta) = -\frac{1}{n} \sum_{i=1}^n H(y_i, h_i)$. Softmax: $\hat{y}_j = e^{\beta z_j} / \sum_{k=1}^K e^{\beta z_k}$ (stable: $z_j \leftarrow z_j - \max \mathbf{z}$; $\beta \to \infty$ gives one-hot). Multi-class CE: $\mathcal{L} = -\frac{1}{n} \sum_{i,k} y_k^{(i)} \log \hat{y}_k^{(i)}$; for one-hot c, $-\log \hat{y}_c$. Entropy $H(P) = -\sum_x P(x) \log P(x)$, cross-entropy $H(P, Q) = -\sum_x P(x) \log Q(x)$, KL $D_{\mathrm{KL}}(P\|Q) = H(P, Q) - H(P) = \sum_x P(x) \log \frac{P(x)}{Q(x)}$. Target P (one-hot/soft), prediction $Q = \hat{y}$; minimizing CE equals minimizing KL.

Regularization

L2: $\ell = \sum \Delta_i^2$ (weight decay; sensitive to outliers; quadratic shrinkage). L1: $\ell = \sum |\Delta_i|$ (robust, constant gradient, induces sparsity). Smooth L1 (Huber): $\mathrm{smooth}_{L1}(x) = 0.5x^2$ if $|x| < 1$, else $|x| - 0.5$. Best of both: robust to outliers + smooth near zero.

4.4 Optimization

Backpropagation

Logistic: $z = \theta^\top x$, $h = \sigma(z)$, $\ell = -[y \log h + (1 - y) \log(1 - h)]$. $\partial\ell/\partial z = h - y$ via $\partial\ell/\partial h = -(y/h - (1-y)/(1-h))$ and $\sigma'(z) = h(1-h)$. $\nabla_\theta \ell = (h - y)x$. Batch: $\nabla_\theta \mathcal{L} = \frac{1}{n} X^\top (H - Y)$.

MLP: $\mathbf{z}^{(\ell)} = \mathbf{W}^{(\ell)} \mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}$, $\mathbf{a}^{(\ell)} = g_\ell(\mathbf{z}^{(\ell)})$. $\boldsymbol{\delta}^{(\ell)} = \partial\mathcal{L}/\partial\mathbf{z}^{(\ell)}$. Output (sigmoid+BCE): $\boldsymbol{\delta}^{(L)} = \mathbf{a}^{(L)} - \mathbf{y}$ because $\partial\ell/\partial a = -y/a + (1-y)/(1-a)$ cancels with $g_L' = a(1-a)$. Hidden: $\boldsymbol{\delta}^{(\ell)} = ((\mathbf{W}^{(\ell+1)})^\top \boldsymbol{\delta}^{(\ell+1)}) \odot g_\ell'(\mathbf{z}^{(\ell)})$. $\partial\mathcal{L}/\partial\mathbf{W}^{(\ell)} = \boldsymbol{\delta}^{(\ell)} (\mathbf{a}^{(\ell-1)})^\top$, $\partial\mathcal{L}/\partial\mathbf{b}^{(\ell)} = \boldsymbol{\delta}^{(\ell)}$.

Optimizers

SGD: $\theta_{t+1} = \theta_t - \alpha g_t$. Momentum: $v_t = \rho v_{t-1} + (1 - \rho) g_t$, $\theta_{t+1} = \theta_t - \alpha v_t$. RMSProp: $s_t = \rho s_{t-1} + (1 - \rho) g_t^2$, $\theta_{t+1} = \theta_t - \alpha \frac{g_t}{\sqrt{s_t} + \epsilon}$. Adam: $m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$, $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$; bias-correct $\hat{m}_t, \hat{v}_t$; update with $\alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$.

Learning Rate

Strategies: fixed; step decay; exponential $\alpha_t = \alpha_0 \gamma^t$; linear; cosine $\alpha_t = \frac{1}{2} \alpha_0 (1 + \cos(t\pi/T))$; warmup; ReduceLROnPlateau. Tip: Adam + fixed decay to find convergence epoch, then SGD + Momentum + cosine.

4.5 Evaluation Metrics

Accuracy= $\frac{TP+TN}{TOT}$; Precision= $\frac{TP}{TP+FP}$; Recall= $\frac{TP}{TP+FN}$; F1= $2\frac{P \cdot R}{P+R}$. Generalization gap: $\mathbb{E}_{\text{test}}[\text{loss}] - \mathbb{E}_{\text{train}}[\text{loss}]$.

5 Convolutional Neural Networks

5.1 CNN Core Mechanisms

Conv Layer

Input $W_{in} \times H_{in} \times C$; kernel $F \times F \times C$; K filters; stride S; padding P. $W_{out} = \lfloor \frac{W_{in} - F + 2P}{S} \rfloor + 1$; params $= (F^2 C + 1)K$.

Padding and Stride

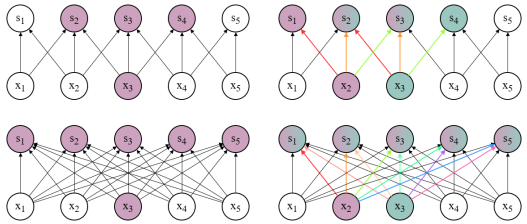
Valid: $P = 0$. Same: $P = \lfloor \frac{F-1}{2} \rfloor$. Full: $P = F - 1$. Content: zero, constant, reflect, replicate, circular.

Pooling

Types: max (default), average, sum. No trainable params. Increases invariance to small trans/rotstion.

CNN vs FC

Params: Conv $(F^2C + 1)K$ vs FC $(W_{in}H_{in}C + 1)(W_{out}H_{out}K)$. Sparse connectivity + parameter sharing give CNN translation equivariance and location preservation. MLP $\supseteq$ CNN in expressive power; CNN adds **inductive bias** for vision.



5.2 CNN Training Practice

Data Preprocessing

Normalize: per-channel zero-mean/unit-var or scale to $[0, 1]$. Augment: random crop, flip, rotate, color jitter. Shuffle and batch.

Weight Initialization

Preserve variance: $\sigma_w^2 = \frac{1}{n_{\text{in}}}$. Xavier/Glorot (tanh/sigmoid): $\sigma_w^2 = \frac{1}{n_{\text{in}}}$ or $\frac{2}{n_{\text{in}}+n_{\text{out}}}$. He (ReLU): $\sigma_w^2 = \frac{2}{n_{\text{in}}}$. Uniform: $W \sim \mathcal{U}(-\sqrt{3\sigma_w^2}, \sqrt{3\sigma_w^2})$. Gaussian: $W \sim \mathcal{N}(0, \sigma_w^2)$.

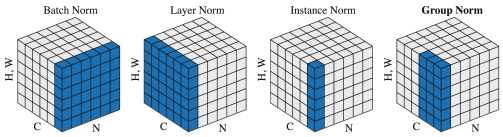
Batch Training

Iteration: one forward+backward on a batch. Epoch: full pass over training set.

Handle Underfitting

Handle Underfitting

Normalization. (1) BatchNorm. Train: Per channel: $\mu = \frac{1}{NHW}\sum x$, $\sigma^2 = \frac{1}{NHW}\sum(x-\mu)^2$, $\hat{x} = \frac{x-\mu}{\sqrt{\sigma^2+\epsilon}}$, $y = \gamma\hat{x} + \beta$ ($2C$ params). Eval: $\mu_{\text{run}} \leftarrow \rho\mu_{\text{run}} + (1-\rho)\mu_B$, σ_{run}^2 likewise; inference with running stats. Before activation, never on output. (2) MLP: per feature dim over N samples ($2D$ params). (3) Pros: stabilizes layers (larger LR), smooths landscape, mild reg. Cons: train-eval gap at small batch; assumes i.i.d. spatially, fails for correlated data. (4) Variants: LayerNorm (per sample); InstanceNorm (per channel per sample); GroupNorm (channel groups). No batch dependence.



Residual Link. $y = F(\mathbf{x}) + \mathbf{x}$. Learn residual; drive $F \to \mathbf{0}$ if depth hurts. Pros: bypass fights vanishing/exploding; flatter landscape.

Handle Overfitting

Match **data variability** to **model capacity**. (1) Early Stopping. Stop when val acc degrades. (2) Data Augmentation. Input transforms; magnitude critical—too strong induce error/ambiguity, too weak useless. (3) Regularization. $\mathcal{L}_{\text{reg}} = \mathcal{L}_{\text{main}} + R(\theta)$; only W penalized. L2: $R = \frac{\lambda}{2}\|W\|_2^2$; weight decay $\alpha\lambda$. L1: $R = \lambda\|W\|_1$; step $\alpha\lambda$ to zero, sparsity. Elastic Net: $R = \lambda_1\|W\|_1 + \frac{\lambda_2}{2}\|W\|_2^2$; both. (4) Dropout. Train: mask $\mathbf{r} \sim \text{Bernoulli}(1-p)$, output $\mathbf{r} \odot \mathbf{a}$. Breaks co-adaptation. Eval: scales outputs by $1-p$. Inverted Dropout: train scale $1/(1-p)$, eval no-op(preferred). FC only; BN suffices for Conv. (5) BN as Regularization. Gaussianizes pre-activations + batch noise, cuts capacity.

5.3 Classification Backbones

Receptive Field

Jump: $j_\ell = \prod_{i=1}^{\ell-1} s_i$, $j_1 = 1$. Receptive field: $f_\ell = f_{\ell-1} + (k_{\ell-1} - 1)j_{\ell-1}$, $f_1 = 1$. $k_\ell \geq s_\ell$ so no gaps.

VGG

Smaller kernels, deeper nets. Three 3×3 layers = one 7×7 RF, but more nonlinearities and fewer params ($27C^2$ vs $49C^2$).

ResNet

Residual block. Bottleneck: 1×1 (reduce), 3×3 (spatial), 1×1 (restore). Bottleneck params $\ll$ Basic block.

Beyond ResNet

SENet: channel attention. DenseNet: dense connections, concat all previous. MobileNet: separable conv. depthwise (per-channel) + pointwise (1×1 channel mix). Cost $\approx$ standard$/N$. NAS: neural architecture search.

6 Semantic Segmentation

6.1 Encoder-Decoder

FCN Encoder

Encoder $E_\theta : \mathcal{X} \to \mathcal{Z}$ downsamples by total factor $D = \prod s_\ell$. Bottleneck size $H_L = \lfloor H_0/D \rfloor$, $W_L = \lfloor W_0/D \rfloor$. Decoder D_ϕ upsamples back to original size. Segmentation output: $\hat{y} \in \mathbb{R}^{K \times H_0 \times W_0}$.

Non-learnable Upsampling

Bilinear interpolate. Nearest neighbor: replicate each pixel to $r \times r$ block. Max unpooling: record maxpool indices, scatter back, zero-fill (so pair with maxpool).

Transposed Convolution

$y = w * x = \mathbf{C}x$, transpose $x = w *^\top y = \mathbf{C}^\top y$. Equivalent ordinary conv: insert $s-1$ zeros between pixels; surround-pad $k-p-1$ zeros; extra bottom/right pad $a = (h_{\text{in}} + 2p - k) \bmod s$ if needed; then conv stride 1. Output: $h'_{\text{out}} = s(h'_{\text{in}} - 1) + a + k - 2p$. Checkerboard artifact when k not divisible by s.

6.2 UNet

Skip Connections

Skip links: concat encoder features to decoder at same resolution, splitting the burden—bottleneck carries **global context**, skips inject **per-pixel spatial detail**. Without them, the bottleneck must encode both, degrading optimization.

6.3 Segmentation Evaluation

Overlap Metrics

IoU$= \frac{TP}{TP+FP+FN} = \frac{|P \cap G|}{|P \cup G|}$. mIoU$= \frac{1}{K}\sum_{k=1}^{K} \text{IoU}_k$ (fair for class imbalance). Pixel accuracy can be misleading when positives are rare.

Soft IoU Loss

Soft Intersection$= \sum_i p_i y_i$; Soft Union$= \sum_i (p_i + y_i - p_i y_i)$. $\mathcal{L}_{\text{IoU}} = 1 - \frac{\sum_i p_i y_i + \epsilon}{\sum_i p_i + \sum_i y_i - \sum_i p_i y_i + \epsilon}$. Multi-class: average per-class soft IoU loss.

7 Object Detection

7.1 Basics

Output axis-aligned bbox (x, y, h, w) plus class confidence score. Backbone CNN extracts feature map.

7.2 Two-Stage Detectors

R-CNN

Selective search proposals, each cropped and fed to CNN independently. Slow, not end-to-end, loses context.

Fast R-CNN

CNN first on whole image to get a feature map, then project selective search proposals onto it and use RoI Pooling to extract fixed-size features. Faster than R-CNN but still uses selective search as a bottleneck.

Faster R-CNN

RPN replaces selective search on shared feature map.

RoI Generation

RPN outputs proposals with objectness scores. Post-process: NMS ($\tau = 0.7$), top-K, positive/negative sampling by IoU thresholds (pos > 0.7, neg < 0.3, ignore middle). RPN anchor regression targets: $t_x = \frac{x_g - x_a}{w_a}$, $t_y = \frac{y_g - y_a}{h_a}$, $t_w = \log\frac{w_g}{w_a}$, $t_h = \log\frac{h_g}{h_a}$. RPN loss: $L = L_{\text{cls}} + \lambda L_{\text{reg}}$ with BCE for cls and Smooth L1 for reg (positive only).

7.3 Single-Stage Detectors

YOLO/SSD: dense prediction on feature map grid $H \times W$ with B anchors per cell. Output $H \times W \times (5B + K)$. Direct cls + reg without proposals. Category-specific vs RPN's foreground/background. Faster inference, slightly lower accuracy historically.

7.4 Post-processing and Evaluation

Non-Maximum Suppression

NMS: sort by score; greedily keep highest, remove IoU$> \tau$. Soft-NMS: decay scores by overlap instead of hard removal. Class-wise NMS: run per category.

Detection Metrics

TP: correct class && IoU$\geq \tau$ && unmatched GT. FP: wrong class || IoU$< \tau$ || duplicate match. FN: unmatched GT. AP: area under PR curve (11-point interpolation). mAP: mean over classes or IoU thresholds.

8 Instance Segmentation

8.1 Mask R-CNN Pipeline

Architecture

Top-down Architecture. Based on Faster R-CNN with added mask head. Three parallel heads: cls, reg, mask. Training: mask head supervised only on positive RoIs.

RoIAlign

Avoid quantization of RoI Pool. (1) keep float proposal locations; (2) divide into bins; (3) sample 4 points per bin with bilinear interpolation; (4) pool (max/avg). Significant AP gain, especially at high IoU thresholds.

8.2 Mask Head Design

Output Strategy

Use per-pixel Sigmoid (independent binary classification) instead of Softmax across classes. Avoids class competition since cls head already decides category; mask head focuses purely on shape.

Channel Strategy

Class-specific: K binary mask channels, params $\propto K$. Class-agnostic: single shared mask. Mask R-CNN: K channels but only the cls-selected one is used for loss and inference.

Loss

Per-pixel binary cross-entropy over $m \times m$ mask (e.g., 28×28):

$$\mathcal{L}_{mask} = -\frac{1}{m^2}\sum_{u=1}^{m}\sum_{v=1}^{m}\left[y_{uv}\log\hat{y}_{uv} + (1-y_{uv})\log(1-\hat{y}_{uv})\right]$$

Only the GT class channel contributes gradients; other channels ignored.